Local Finance Tracker — Documentation

Project overview

Local Finance Tracker is a privacy-focused, offline-first personal finance command center built with Python, Pandas, and Streamlit. It extracts and normalizes transactions from:

- Bank statement PDFs
- Android SMS XML backups

All processing runs locally by default, with optional Google integrations for backup syncing and summary exports.

Key features

- Offline-first transaction processing (local machine)
- Coordinate-based PDF extraction for reliable statement parsing (PyMuPDF / `fitz`)
- SMS XML parsing with spam filtering (regex-based)
- Pandas-based cleanup, pivots, and monthly/yearly analytics
- Streamlit dashboard with guided “wizard” start flow
- Optional Google Drive import + Google Sheets export

Architecture

The project is organized into a Streamlit frontend and a backend processing pipeline.

Frontend: `app.py`

- Streamlit dashboard with a wizard-style landing page to select a data source
- Views include:
  - Yearly overviews
  - Monthly spend vs. credit charts (side-by-side)
  - Transaction ledger showing positive (credit) and negative (debit) entries

Backend pipeline

- Parser: `src/parser.py`
  - PDF parsing
    - Uses PyMuPDF (`fitz`) with coordinate-based extraction
    - Designed to handle “invisible gridlines” and multi-line transaction descriptions reliably
  - SMS XML parsing
    - Parses Android SMS backup XML
    - Uses regular expressions to filter spam and extract:
      - Amount
      - Transaction type (`DEBIT` / `CREDIT`)
- Analyzer: `src/analyzer.py`
  - Uses Pandas for data cleaning and normalization
  - Builds pivot tables and aggregates trends by month and year
- Cloud integrations (optional)
  - Drive connector: `src/drive_api.py`
    - Fetches the latest SMS XML from Google Drive
  - Sheets connector: `src/sheets_api.py`
    - Exports the finalized financial summary to Google Sheets

File naming and storage conventions

For the automated pipeline to work correctly, raw input files must be placed with exact names.

Required folder

- Store all raw input files in: `data/raw/` (at the project root)
- If the folder does not exist:
  - Create it manually, or
  - Use the app's "Manual Upload" option (which will create it automatically)

Required filenames

- Bank statement PDF: `statement.pdf`
- SMS backup XML: `sms_backup.xml`

Exporting SMS data from Android (XML)

1. Install "SMS Backup & Restore" from the Google Play Store.
2. Open the app and choose "Set Up a Backup".
3. Disable "Call Logs" and ensure only "Messages" is selected.
4. Choose a backup destination (Google Drive recommended).
5. Run the backup.
6. If the XML is saved locally on the phone:
   - Connect the phone to your computer
   - Copy the backup `.xml` file into `data/raw/`
   - Rename it to `sms_backup.xml`

Google API setup (optional)

These steps are only required if you want to use:

- "Fetch from Drive" (Google Drive import)

- “Sync to Sheets” (Google Sheets export)

All data remains private to your Google account.

Enable required APIs

1. Open Google Cloud Console: https://console.cloud.google.com/
2. Create a new project (example: “Finance Tracker”).
3. Go to APIs & Services → Library.
4. Enable:
    - Google Drive API
    - Google Sheets API

Create OAuth credentials

1. Go to APIs & Services → Credentials.
2. Select Create Credentials → OAuth client ID.
3. Set Application type to “Desktop app”.
4. Download the JSON credentials file.

Store credentials securely

1. Rename the downloaded file to: `credentials.json`
2. Place `credentials.json` in the project root (next to `app.py`).
3. On first use, a browser window will open for Google sign-in and consent.
4. After approval, the app will generate `token.json` automatically to store the session.

> ⚠️ Security warning
>
> - Never share `credentials.json` or `token.json`.
> - Add both files to `.gitignore` before pushing to GitHub.

Setup (from GitHub)

1) Clone the repository

```
git clone https://github.com/YourUsername/local-finance-track
er.git
cd local-finance-tracker
```

2) Create and activate a virtual environment

```
# Create the environment
python -m venv venv

# Activate on Windows:
.\venv\Scripts\activate

# Activate on Mac/Linux:
source venv/bin/activate
```

3) Install dependencies

```
pip install -r requirements.txt
```

4) Run the app

```
streamlit run app.py
```

Troubleshooting

- If no transactions appear, confirm:
  - `data/raw/statement.pdf` exists (for PDF mode)
  - `data/raw/sms_backup.xml` exists (for SMS mode)
  - Filenames match exactly (case-sensitive on some systems)

- If Google sync fails, confirm:
  - `credentials.json` is in the project root
  - You completed the OAuth browser consent flow
  - Both Drive and Sheets APIs are enabled in the selected Google Cloud project

Contact

If there are any issues in the code, contact me.

- Email: **meiygowri2006@gmail.com**